DESENVOLVIMENTO DE APLICATIVOS MOBILE



MÓDULO 11

Testes e Qualidade

Pirâmide de testes, testes unitários com coroutines, fakes e mocks, testes de UI em Compose, desempenho, acessibilidade e integração contínua

Prof. Dr. Francisco
IFMA – Campus São Luís – Monte Castelo
Departamento de Eletroeletrônica (DEE)

1 de novembro de 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Planejar testes segundo a pirâmide de testes.
- Testar funções, casos de uso e ViewModels com JUnit, coroutines e Turbine.
- Usar *fakes* e *mocks* para isolar dependências.
- Escrever testes de interface em Compose e testes de banco com Room.
- Diagnosticar problemas de desempenho e recomposição.
- Tornar o app acessível e automatizar a verificação com integração contínua.

Roteiro do módulo

1. Estratégia de testes
2. Testes unitários
3. Testes de integração e de UI
4. Desempenho
5. Acessibilidade
6. Qualidade contínua
7. Fechamento

1
Estratégia de testes

A pirâmide de testes



- **Unitários** (`src/test`): muitos, rápidos, rodam no computador.
- **Integração**: várias peças juntas (DAO + banco em memória).
- **UI / ponta a ponta** (`src/androidTest`): poucos, verificam fluxos críticos no aparelho.

O que testar primeiro

Regras de negócio (casos de uso), transformações de dados e os ViewModels. É onde estão os erros mais caros.

Arquitetura testável

As camadas, as interfaces e a injeção de dependência dos Módulos 06–07 existem justamente para permitir isso.

2
Testes unitários

Primeiro teste: funções puras

src/test/.../ConversoesTest.kt

```
class ConversoesTest {

  @Test
  fun `zero celsius é 32 fahrenheit`() {
    assertEquals(32.0, 0.0.cParaF(), 0.001)
  }

  @Test
  fun `adc de 12 bits no fundo de escala vale vref`() {
    assertEquals(3.3, 4095.adcParaVolts(), 0.001)
  }

  @Test
  fun `leitura fora da faixa gera alerta`() {
    val alertas = VerificadorFaixa(10.0..40.0)
      .verificar(listOf(25.0, 41.2, 9.0))
    assertEquals(listOf(41.2, 9.0), alertas)
  }
}
```

Estrutura AAA

Arrange (prepare os dados), **Act** (execute) e **Assert** (verifique). Um comportamento por teste.

Nomes legíveis

Kotlin aceita nomes de função entre crases: o relatório de testes vira uma especificação em português.

Executando

```
./gradlew test
# relatório em app/build/reports/tests/
```

Testando coroutines

MainDispatcherRule.kt

```
// Substitui Dispatchers.Main (inexistente na JVM)
class MainDispatcherRule(
  val dispatcher: TestDispatcher =
    UnconfinedTestDispatcher()
) : TestWatcher() {
  override fun starting(d: Description) =
    Dispatchers.setMain(dispatcher)
  override fun finished(d: Description) =
    Dispatchers.resetMain()
}
```

Teste com runTest

```
@Test
fun `sincronizar tenta de novo após falha`() = runTest {
  val api = FakeApi(falhasAntesDeResponder = 2)
  val repo = BancadaRepositoryImpl(FakeDao(), api)
  repo.sincronizarComRetry()   // delays são "pulados"
  assertEquals(3, api.chamadas)
}
```

runTest

Executa código `suspend` em testes com **tempo virtual**: um `delay(60_000)` termina instantaneamente.

Dispatchers injetados

Repositórios que recebem o dispatcher no construtor (Módulo 03) podem receber um `StandardTestDispatcher` no teste.

Dependência

`kotlinx-coroutines-test` em `testImplementation`.

Fakes *versus* mocks

Fake: implementação simples

```
class FakeBancadaRepository : BancadaRepository {
  private val dados =
    MutableStateFlow<List<Bancada>>(emptyList())
  var falharSync = false

  fun emitir(lista: List<Bancada>) { dados.value =
      lista }

  override fun bancadas() = dados
  override suspend fun sincronizar() {
    if (falharSync) throw IOException("sem rede")
  }
  override suspend fun reservar(id: Int, ate: Instant)
      {}
}
```

Mock com MockK

```
val api = mockk<LabApi>()
coEvery { api.listarBancadas() } throws
      IOException()

val repo = BancadaRepositoryImpl(FakeDao(), api)
val r = runCatching { repo.sincronizar() }

assertTrue(r.isFailure)
coVerify(exactly = 1) { api.listarBancadas() }
```

Quando usar

Fake: reutilizável, comportamento realista; preferido pelo Google. **Mock**: verificar *interações* (foi chamado? quantas vezes?).

Testando o ViewModel com Turbine

PainelViewModelTest.kt

```
class PainelViewModelTest {
  @get:Rule val main = MainDispatcherRule()
  private val repo = FakeBancadaRepository()
  private val vm = PainelViewModel(repo)

  @Test
  fun `filtro mostra apenas bancadas correspondentes`() =
    runTest {
      repo.emitir(listOf(Bancada(1, "Eletrônica", 24.0),
                         Bancada(2, "Máquinas", 22.0)))
      vm.uiState.test {
        assertEquals(2, awaitItem().bancadas.size)
        vm.onFiltroChange("Máq")
        val estado = awaitItem()
        assertEquals(1, estado.bancadas.size)
        assertEquals("Máquinas", estado.bancadas[0].titulo)
        cancelAndIgnoreRemainingEvents()
      }
    }
}
```

Turbine

Biblioteca da Cash App que coleta um `Flow` e permite verificar cada emissão em sequência com `awaitItem()`.

Dica

Teste o **estado** que o ViewModel expõe, não seus detalhes internos. Assim o teste sobrevive a refatorações.

Atenção

`stateIn(WhileSubscribed)` só calcula com alguém coletando. O `test {}` do Turbine é esse coletor.

3
Testes de integração e de UI

Room com banco em memória

src/androidTest/.../BancadaDaoTest.kt

```
@RunWith(AndroidJUnit4::class)
class BancadaDaoTest {
  private lateinit var db: AppDatabase
  private lateinit var dao: BancadaDao

  @Before fun criar() {
    db = Room.inMemoryDatabaseBuilder(
      ApplicationProvider.getApplicationContext(),
      AppDatabase::class.java).build()
    dao = db.bancadaDao()
  }
  @After fun fechar() = db.close()

  @Test fun salvarEObservar() = runTest {
    dao.salvar(BancadaEntity(1, "B1", "L-12"))
    val lista = dao.observarTodas().first()
    assertEquals("B1", lista.single().nome)
  }
}
```

Banco em memória

Criado do zero a cada teste e descartado no fim: testes independentes e rápidos, sem afetar dados reais.

Migrações

`MigrationTestHelper` cria o banco na versão antiga, aplica a migração e valida o esquema resultante.

Dica

Os testes em `androidTest` rodam com `./gradlew connectedAndroidTest` em um aparelho ou emulador.

Testes de interface em Compose

TelaPainelTest.kt

```
class TelaPainelTest {
  @get:Rule val compose = createComposeRule()

  @Test fun mostraAlertaQuandoAcimaDoLimite() {
    compose.setContent {
      CartaoBancada(numero = 3, temperatura = 35.0)
    }
    compose.onNodeWithText("Acima do limite!")
      .assertIsDisplayed()
  }

  @Test fun clicarNaBancadaChamaCallback() {
    var aberta = -1
    compose.setContent {
      TelaPainel(estadoExemplo, {}, {}, onAbrir = { aberta =
      it })
    }
    compose.onNodeWithTag("bancada-2").performClick()
    assertEquals(2, aberta)
  }
}
```

Encontrar, agir, verificar

Encontrar: `onNodeWithText`, `onNodeWithTag`, `onNodeWithContentDescription`.
Agir: `performClick`, `performTextInput`.
Verificar: `assertIsDisplayed`, `assertExists`.

Telas sem estado

A divisão *Route*/*Screen* do Módulo 06 permite testar a tela passando qualquer `UiState`, sem ViewModel nem rede.

4
Desempenho

Diagnosticando desempenho

Ferramentas

- **Profiler** do Android Studio: CPU, memória, energia; identifica *jank* (quadros perdidos).
- **Layout Inspector**: contagem de recomposições por composable.
- **Macrobenchmark**: mede inicialização e rolagem de forma reproduzível.
- **Android vitals** (Play Console): ANRs e falhas nos aparelhos reais.

Atenção

Meça sempre em build **release** e em aparelho real: o build debug é muito mais lento.

Compose rápido

- Nada de trabalho pesado durante a composição: formate e filtre no ViewModel.
- `key` estável em listas *lazy*.
- `derivedStateOf` para estados que mudam com frequência (ex.: posição de rolagem).
- Classes de estado imutáveis (`val`, `List`).

Baseline Profiles

Arquivo que lista o código usado na inicialização. A Play Store pré-compila esse código e o app abre bem mais rápido.

5
Acessibilidade

Apps para todas as pessoas

Semântica em Compose

```
// imagem informativa: descreva
Icon(Icons.Default.Warning,
  contentDescription = "Temperatura acima do limite")

// card inteiro lido como um único item
Card(Modifier.semantics(mergeDescendants = true) {}) {
  Text("Bancada 3"); Text("24,8 °C")
}

// estado anunciado pelo leitor de tela
Switch(checked = ligado, onCheckedChange = { ligado =
      it },
  modifier = Modifier.semantics {
    stateDescription = if (ligado) "ligado" else
      "desligado"
  })
```

Checklist

- `contentDescription` em ícones e imagens informativas.
- Alvos de toque de pelo menos **48 dp**.
- Contraste mínimo de **4,5:1** para texto.
- Funciona com fonte em **200 %** (use `sp`).
- Informação não depende só de cor (verde/vermelho + ícone ou texto).
- Navegável com **TalkBack** e por teclado.

Dica

O app **Accessibility Scanner** analisa a tela aberta e aponta problemas automaticamente.

6
Qualidade contínua

Análise estática e integração contínua

`.github/workflows/android.yml`

```yaml
name: Android CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew lint testDebugUnitTest
      assembleDebug
      - uses: actions/upload-artifact@v4
        with:
          name: apk-debug
          path: app/build/outputs/apk/debug/*.apk
```

Ferramenta	Verifica
Android Lint	APIs incorretas, permissões, recursos não usados, acessibilidade
ktlint	formatação e estilo Kotlin
detekt	complexidade e más práticas
Testes	comportamento

Integração contínua

A cada *push*, um servidor compila, analisa e testa o projeto. Um *pull request* só é aceito com tudo verde.

Dica

Cheque as versões atuais das *actions* no GitHub Marketplace antes de usar.

7
Fechamento

Resumo

Resumo

- Muitos testes unitários, alguns de integração e poucos de UI.
- `runTest` + `MainDispatcherRule` para coroutines; Turbine para `Flow`.
- Fakes para simular dependências; mocks (MockK) para verificar interações.
- Room em memória e `createComposeRule` para integração e interface.
- Desempenho: meça em release, no aparelho, com Profiler, Layout Inspector e Baseline Profiles.
- Acessibilidade e lint fazem parte da qualidade; a CI verifica tudo a cada *push*.

Exercícios

Exercício 1 — Unitários

Escreva testes para `VerificarAlertasUseCase` cobrindo: lista vazia, todas abaixo do limite, uma acima, temperatura nula e mudança do limite.

Exercício 2 — ViewModel

Teste o `PainelViewModel` para os estados carregando, sucesso, vazio e erro de sincronização usando `FakeBancadaRepository` e Turbine.

Exercício 3 — Acessibilidade

Rode o Accessibility Scanner nas telas do LabMonitor, navegue com TalkBack e corrija ao menos três problemas encontrados.

LabMonitor — Etapa 11

Atinja ao menos 70 % de cobertura nos casos de uso e ViewModels, escreva dois testes de UI para os fluxos de reserva e configure o GitHub Actions para rodar lint e testes.

Para aprofundar

- Testes no Android: `https://developer.android.com/training/testing`
- Testando coroutines: `https://developer.android.com/kotlin/coroutines/test`
- Testes em Compose: `https://developer.android.com/develop/ui/compose/testing`
- Turbine: `https://github.com/cashapp/turbine` MockK: `https://mockk.io`
- Desempenho em Compose: `https://developer.android.com/develop/ui/compose/performance`
- Baseline Profiles: `https://developer.android.com/topic/performance/baselineprofiles`
- Acessibilidade: `https://developer.android.com/guide/topics/ui/accessibility`

Fim do Módulo 11

Próximo: Módulo 12 — Publicação na Google Play: assinatura, AAB, Play Console, testes fechados, LGPD e CI/CD.